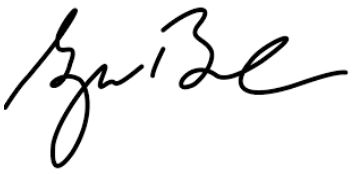
General Style and Coding Standards for Software Projects

Preliminary Version

Author(s): Standards Group Date: March 20, 2015	Approved:  Director of IT department Date: March 26, 2015	Revised by: Date:

INTRODUCTION

PURPOSE

The goal of this guideline is to create uniform coding habits among software personnel in the software engineer team so that reading, checking, and maintaining code written by different persons becomes easier. The intent of these standards is to define a natural style and consistency of the source codes created by software engineer team.

When a project adheres to common standards many good things happen:

- Programmers can go into any code and figure out what is going on, so maintainability, readability and reusability are increased. Code walk through become less painful.
- New people can get up to speed quickly.
- People new to a language are needed to develop a personal coding style that complies with the team standard.
- People make fewer mistakes in consistent environments.
- Coding standard help increasing productivity, maintainability for the source codes.
- Coding standard help the project run smoothly.
- Make the software development environment be flexible, control the ego a bit, and remember any project is a team effort.

Most arguments against a particular standard come from the personal ego. So, to use coding standard could be flexible, control the personal ego a bit, and remember any project is a team effort.

“A mixed coding style is harder to maintain than a bad coding style”. So, it is important to apply a consistent coding style across a project.

After delivery, maintenance or enhancement, coding standard reduce the cost of project by easing the learning or re-learning task when code needs to be addressed by people other than the author after long absence.

Naming Variables

Clear, complete, and meaningful names make the code more readable and minimize the need for comments.

Names of variable should be “noun” with or without modifier and use Prefix to identify data type of that variable.

[return type]_[noun] *[return type]_[adjective]* *[return type]_[adjective]_[noun]*

Example:

variable name	return type	adjective	noun
dt_time	date time		time
dt_current_time	date time	current	time
i_student_total_num	integer		student_totol_number
f_gpax	float		gpax

Naming Convention for Functions

Function without return type:

Name of functions shall consist of a verb and (whenever appropriate) an object

[verb]_[object]

Example:

function name	verb	Object
Enter_Height_and_Weight	Enter	Height and Weight
Calculate_GPAX	Calculate	GPAX

Function with return type:

Name of functions should use “Prefix” to identify the return type on functions that return value.

[return_type]_[verb]_[noun]

Example:

function name	return type	verb	noun
i_Get_Time	Integer	Get	Time
bln_Create_OutputFile	boolean	Create	OutputFile

Naming Constants

Names of constants should be noun, capital letter with or without modifiers.

[adjective]_[noun]

Example:

variable name	adjective	noun
MAX_LINES	MAX	LINES
PI		PI
AVERAGE_GPA	AVERAGE	GPA

Naming Boolean variables

Name of Boolean variable should be start with return type, then verb and object or adjective.

[return_type]_[verb]_[noun]_[adjective]

Example:

variable name	return type	verb	Adjective/noun
bln_Save_Completed	Boolean	Save	Completed
bln_Is_File_Opened	Boolean	Is	File Opened

Naming Types

All names of types use the letters “Type”

[noun]_Type

Example:

variable name	noun
subject_Type	subject
license_Type	license
bankAccount_Type	bank account

Identifying variable types

Type	Prefix
Boolean	bl, bln
Char	ch
Unsigned char	uc
Short	s
Unsigned short	us
Long	l
Unsigned long	ul
Float (or single)	f
Double	d
Bit	b
Byte	by
String	str

Conditionals and comparisons

- When use IF.., always use Else

Java Example of a Helpful, Commented *else* Clause

```
// if color is valid
if ( COLOR_MIN <= color && color <= COLOR_MAX ) {
   // do something
   ...
}
else {
   // else color is invalid
   // screen not written to -- safely ignore command
}
```

- Always put comment before starting conditional or loops, and comment for each calling external functions with explain the input parameters and returned output.

```
// comment objective of this IF
if (boolean variable == TRUE/FALSE)
{
    do_This(); // comment objective of this function
    bln_flag = do_That();  // comment objective of this function, what's value to be returned
}
// comment objective of this else
else
{
    //comment objective of this function, what's values to be passed to this function
    do_This(i_max_Score, str_Student_Name);
    do_That();    // comment objective of this function
}
```

Error Handling

- Use Try..Catch..and Finally as always inside the program
- Catch exception exactly match with the possible occurred exceptions
- All line of code throughout the program has to be under Try..Catch Block or within the throw exception.

```
try {
    * Program create external file=> output file can not create!
    * For Loop uses size of Array => Array may not be initiate, no size!

} catch (ArrayIndexOutOfBoundsException e) {
      System.err.println("Caught  ArrayIndexOutOfBoundsException:  " + e.getMessage());
} catch (IOException e) {
  System.err.println("Caught  IOException: " + e.getMessage());
}
```

```
public static void calcSqrt(String[] inputValue) throws IndexOutOfBoundsException,NumberFormatException {
        double d_inputValue = Double.parseDouble(inputValue[0]);
        System.out.println ("Sqrt "+ d_inputValue+ " = "+ Math.sqrt(d_inputValue));
}
```

Function/Procedure

- Put comment before starting function or procedure to describe objective of this function, input parameters, returned value of function.

```
/* Function Objective:
   Input parameters :
   Function description:
   Return Value : [void, int, float, string] GPA value
   Created By : Programmer A
   Create Date:
   Revised By:
   Revised Date:
*/
void Function_Name (parameter1, parameter2) {
  // validate parameter1, parameter2 with "assertion" OR..
  // validate parameter1, parameter2 with "error message"


}
```

- cxcx